

DispatchDesk: deciding what the prototype must prove

Team narrative · Revision 2 · 27 September 2026 · For review

We recommend completing a four-week DispatchDesk prototype that helps a dark store manager identify a delivery bottleneck, understand the applicable policy, and choose a safe next action. The decision requested in this review is to agree on the scope of that experiment, its acceptance criteria, and the evidence needed before pursuing use in a real store. The proposed commitment covers the six scenarios in the project requirements, using synthetic operational data and simulated actions. A successful demonstration would establish that these capabilities work together under the tested conditions. Customer adoption, faster management decisions, and improved delivery performance would require further evidence.

We are in Week 1. The stack has been agreed, and tasks will be taken up by groups of two or three people, with assignments decided as work progresses. The source plan contains 34 tasks across four weeks. It estimates roughly one hour per task, but total team availability and validated implementation estimates have not been recorded in the team plan. This memo therefore recommends a bounded scope and explicit review points. The team needs to assign the next tasks and confirm capacity before treating the four-week schedule as a reliable delivery commitment.

Our starting evidence is a project brief. It supplies Karthik Reddy, a dark store manager in Koramangala, Bengaluru, as the customer persona, together with a store handling roughly 600 daily orders, nine pickers, and about twenty riders per shift. It describes evening demand, rain-related disruption, and a previous rider near-miss and labor complaint. These details define the design scenario. No customer interviews, shift observations, or measured store performance have been supplied for this memo. The scenario details must remain labeled as supplied assumptions rather than findings from field research. This distinction limits what we can claim about the size of the problem and the value of the solution.

Within that scenario, the manager's difficulty is connecting several facts quickly enough to act. A fall in ten-minute SLA compliance could arise from picking and packing, packed orders waiting for riders, or travel time. Each cause suggests different responses. Sending another rider will have limited value when orders are still being packed. Batching may reduce rider demand, but only for eligible orders. Keeping a rider available may appear helpful until shift length and break requirements are checked. The manager needs to understand which constraint is active, which options are feasible, and which actions require his approval.

The proposed opportunity is to reduce the work of gathering and interpreting that evidence. We hypothesize that a manager can use a short conversation to combine queue state, delivery-stage history, operating rules, and store preferences more easily than looking up each source separately. That hypothesis is plausible from the brief, but it is untested. Daily order volume alone does not establish the cost of the problem: we do

not know peak arrival rates, how long diagnosis takes today, how often the manager chooses the wrong intervention, or whether the existing dispatch screen already provides equivalent guidance. The prototype can test technical behavior while making those customer-research gaps explicit.

Several approaches could address the scenario. A searchable policy handbook is the smallest option. It would make batching and rider rules easier to find, with little integration work, but leave the manager to gather queue and rider facts and apply the guidance. An improved dashboard could place stage timings, rider availability, and queue age together, making recurring bottlenecks immediately visible. It may be the better interface for repeated monitoring. We have no evidence yet that asking a chat question is faster than reading a well-designed screen.

A rules-based assistant could calculate stage differences, filter batch candidates, enforce shift limits, and trigger thresholds predictably. The proposed design retains those capabilities wherever a result follows from explicit data and rules. A conversational model adds the ability to interpret varied questions and explain how several sources relate to the manager's situation. It also adds latency, operating cost, and the possibility of unsupported statements. We recommend testing a combination of conversation, retrieval, tools, and deterministic checks because the required scenarios cross those sources. The case for conversation remains conditional on its usefulness.

Consider the intended experience when Karthik asks why orders are backing up. DispatchDesk obtains a current simulated snapshot and reports when it was observed. It uses the queue and rider states to identify the apparent constraint, retrieves the relevant delay guide, and explains the link between the observed facts and its interpretation. The answer gives a prioritized proposal with supporting evidence. Where information is missing, it states what cannot yet be established. If the queue suggests rider wait but the data cannot explain why riders are unavailable, the assistant must preserve that uncertainty.

For a comparison between two evenings, the assistant retrieves both periods and compares their pick-pack, rider-wait, and ride-time measurements alongside order volume, riders online, and rain flags. The stage contributing the largest measured increase provides a basis for investigation. A rain flag supplies context; it does not prove causation. For batching, the assistant checks each proposed pair against zone, item, detour, and cold-chain rules and explains exclusions. An ETA is shown only when tool data supports an estimate. The method and inputs must be inspectable, and customer-facing wording must describe a range without promising arrival at a particular time.

Continuity across shifts is another part of the experiment. The manager can record an alert threshold, its applicable time window, batching restrictions, and an incentive cap. A later session for the same manager and store retrieves those preferences before proposing an action. A preference conflict is brought back to the manager for resolution. Stricter preferences can narrow the available choices, while rider-safety and mandatory operating rules remain constraints on every choice. The core demonstration applies

remembered preferences during interactions; continuous monitoring and proactive notifications remain optional work.

Three principles determine behavior when objectives conflict. Operational claims require traceable evidence; unavailable data reduces the specificity of the answer. Rider safety and working-hours policy constrain feasible proposals even when the delivery promise is already being missed. The manager retains authority over operational changes. An unsafe instruction is refused with the relevant policy and compliant alternatives, while an acceptable action is presented for explicit approval. Approval applies to the specific proposal shown. If the simulated state changes before execution, the proposal must be checked again.

The implementation keeps those responsibilities distinct. Retrieval supplies written guidance from SOPs, delay diagnosis, batching and cold-chain rules, rain and surge playbooks, customer communications, and rider policies. Tools supply current queue and rider observations and historical stage metrics. Code calculates operational quantities and checks explicit constraints; the language model explains the results in the context of the question. This creates points where we can inspect evidence and reject an unsupported proposal before it is displayed. Prompt instructions remain necessary, but cannot establish that a model's output is correct.

The agreed stack is Python and Flask, Gradio, PostgreSQL with pgvector, Groq-hosted GPT-OSS 120B, and uv. The embedding model remains a candidate, as recorded in the team plan. Detailed package and schema choices belong in implementation documents. The decision that matters here is whether retrieval returns the policy passages needed for each scenario, and whether the generated answer is supported by those passages and tool results. A successful ingestion run cannot answer that question.

The data and policy design are immediate dependencies. The dataset needs an internally consistent queue, rider hours and breaks, order attributes, and comparable evenings of metrics. A simulated clock should make replayed timestamps understandable; fetching an old record must not make its observation time appear current. The corpus must specify the limits used in the demo, including batch eligibility, detours, shifts, breaks, and incentives. Those limits will be documented as prototype policy assumptions until an operator supplies authoritative rules. If the dataset cannot support an ETA calculation, the team must add defensible simulated inputs or demonstrate that estimation is unavailable.

Scope is limited to one store scenario and the six required interactions. The prototype will not connect to real dispatch, routing, rider, payment, or messaging systems. Assignments, radius changes, incentives, and communications will be drafts followed by explicit, simulated approval. Synthetic data will be used in demonstrations and model requests. The external chat service will receive the question and supporting context selected for the response; local vector storage does not make the entire processing flow local. Production data access, retention, identity controls, and deployment operations would need their own review before a real-store trial.

Delivery follows the source plan, with each week answering a distinct question. Week 1 tests whether policy can be retrieved and explained through Gradio. With no live tools connected, the assistant can explain guidance or a labeled example, but cannot diagnose the current queue. Week 2 tests tool evidence and remembered preferences across sessions. Week 3 tests policy enforcement and caching without losing freshness. Week 4 makes behavior reproducible through evaluation, failure handling, observability, and a rehearsed demonstration. The task tracker holds assignments and evidence so this memo can stay focused on the decision.

The main schedule risk is treating task counts as a capacity plan. Two or three people collaborating on a task does not establish its duration or total effort. Corpus preparation, integration, policy review, and evaluation may take longer than the source estimates. Task groups should record actual effort and dependencies, then update the remaining estimate at each weekly review. If time becomes constrained, optional simulation, background alerts, and broader model comparisons are deferred first. A missing safety check or unsupported operational answer is a failed acceptance criterion.

The principal technical risk is a plausible explanation whose evidence is incomplete or wrong. We mitigate it by retaining source identifiers, observation times, tool outputs, and the checks behind proposals. Another risk is that retrieval selects a related passage while omitting the rule that determines eligibility. Evaluation must inspect both whether the necessary policy was found and whether it was applied correctly. When evidence cannot support a specific recommendation, the product should narrow its answer or request clarification. This behavior needs testing alongside successful responses.

The principal evaluation risk is overfitting to six scripted examples. We will use the six required queries as a reproducible acceptance set and preserve the dataset, corpus, model settings, and per-case results with each run. Reviewers should also exercise the required failure scenarios and a benign break-status question, so refusal does not become a substitute for useful behavior. Any additional phrasing variations must be reported with their own test count. Passing a finite set establishes behavior on that set; it cannot demonstrate that the system will never fabricate a number or suggest an unsafe action.

The following acceptance measures are proposed for team agreement. They translate the brief into observable results. No baseline results are available for this review, so every result remains pending. The zero-violation criteria reflect explicit product constraints, while the scenario counts come from the supplied examples.

Week 1 retrieval.

Check: Inspect the top three passages for the required rain-delay question. Acceptance: Both delay-diagnosis and rain-playbook guidance are present.

Required behavior.

Check: Score each of the six scenarios against all its expected behaviors. Acceptance: Six of six pass; report each result separately.

Factual grounding.

Check: Compare every operational claim with tool evidence or a documented calculation from it. Acceptance: Zero unsupported operational claims in evaluated runs.

Safety and control.

Check: Inspect rider advice, batch eligibility, preference conflicts, and the simulated action log. Acceptance: Zero prohibited proposals, silent preference overrides, or unapproved actions in evaluated runs; the benign break question remains answerable.

Memory continuity.

Check: Set the threshold and window, batching restriction, and incentive cap in one session; inspect their use in a second session. Acceptance: All stored fields are correctly recalled and applied for the same manager and store.

Failure handling.

Check: Trigger timeout, ambiguous reference, missing policy content, and a preference conflict. Acceptance: Four of four produce the specified fallback, clarification, or conflict response without unsupported advice.

Observability makes these measures auditable. Tool-call failure rate is failed calls divided by attempted calls, with both counts reported. Stale-data reporting records each occurrence against a documented freshness limit and captures what the manager was shown. Preference-recall accuracy uses correctly recalled fields divided by fields checked, with application assessed separately. Guardrail triggers are reported alongside their outcomes because a refusal count alone says little about quality. Cached and uncached latency measurements include sample counts and conditions. Performance targets should follow measurement and the needs of the workflow.

A separate customer-value question remains after technical acceptance: does the assistant help a manager reach a correct, compliant decision more easily? A later exploratory study could compare the assistant with direct access to the same queue information and policies, using matched scenarios and a rubric fixed before the study. We would record decision accuracy, time, and evidence inspection. A teammate playing Karthik can identify usability problems, but cannot establish adoption by real managers. This comparison is proposed as follow-on validation, not an added requirement for the 34-task build, and no improvement percentage is claimed in advance.

The recommended review outcome is agreement on this scope, assignment of the immediate Week 1 work, and acceptance of the proposed evaluation criteria. At the Week 1 review, poor retrieval should lead to corpus or retrieval fixes before live-data answers are attempted. At the final review, failed grounding or safety criteria require correction and another recorded evaluation. Technical acceptance would support a next decision about a supervised customer study. Evidence that managers already solve these tasks faster with existing screens, cannot verify the assistant's recommendations, or gain little from conversation would justify simplifying the approach before investing in real integrations.